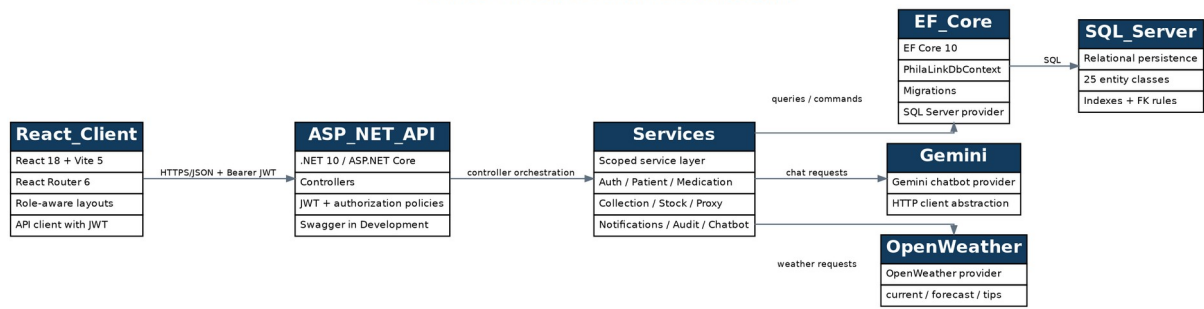


PHILALINK

Verified Project Architecture and Entity Relationship Model

Source-grounded technical reference

Verified PhilaLink runtime architecture



Verification basis

This revision was rebuilt from the connected backend and frontend GitHub source, plus the current PhilaLink project chat for in-progress status. Code is treated as the source of truth when README text conflicts with implementation.

Backend repository: Syabonga-dev/PhilaLink

Frontend repository: Syabonga-dev/PhilaLink_Frontend

Verification date: 16 September 2026

1. Verification findings

The earlier architecture description was broadly correct, but this audit found several places where wording or repository documentation could mislead. This edition separates verified implementation facts from stale documentation and from work that was only discussed in chat.

Claim	Verified status	Evidence / correction
Frontend is a React single-page application	Technically true, wording refined	One index.html, React 18, React Router routes. This PDF uses "React web application with client-side routing" to avoid implying there is only one UI screen.
Backend database is PostgreSQL / Supabase	False for current code	Program.cs calls UseSqlServer and the csproj references Microsoft.EntityFrameworkCore.SqlServer. The README text describing PostgreSQL/Npgsql is stale.
Backend is .NET 10 + ASP.NET Core Web API	True	TargetFramework net10.0, controllers, JWT bearer authentication, Swagger/OpenAPI.
Role model has Patient, Nurse, Proxy, ClinicAdmin, SuperAdmin	True	Role-based routes and backend authorization policies use these five application roles.
Medication supply calculation exists in backend	True	MedicationSupplyController exposes GET /api/medications/me/supply and calculates remaining quantity from latest completed collection, active schedules, and UnitsPerDose.
Accurate supply is fully displayed on dashboard	Not verified in current frontend default branch	Backend endpoint exists. Frontend wiring was the active next development step in project chat; do not present it as completed until pushed and runtime-tested.
All ERD relationships in this report map to persisted entity FKs	True for diagrams shown	Relationships were reconstructed from entity FK/navigation properties and DbContext configuration. Derived DTOs such as MedicationSupplyDto are intentionally excluded from the ERD.

Source precedence

- 1. Current implementation files and EF Core model configuration.
- 2. Current default-branch repository structure.
- 3. Project chat only for development status that is not yet represented in GitHub.
- 4. README documentation only where it agrees with code.

Important correction

The current checked-in backend uses SQL Server, not PostgreSQL. The README still contains PostgreSQL/Npgsql/Supabase language, so that portion should be updated separately.

2. System architecture

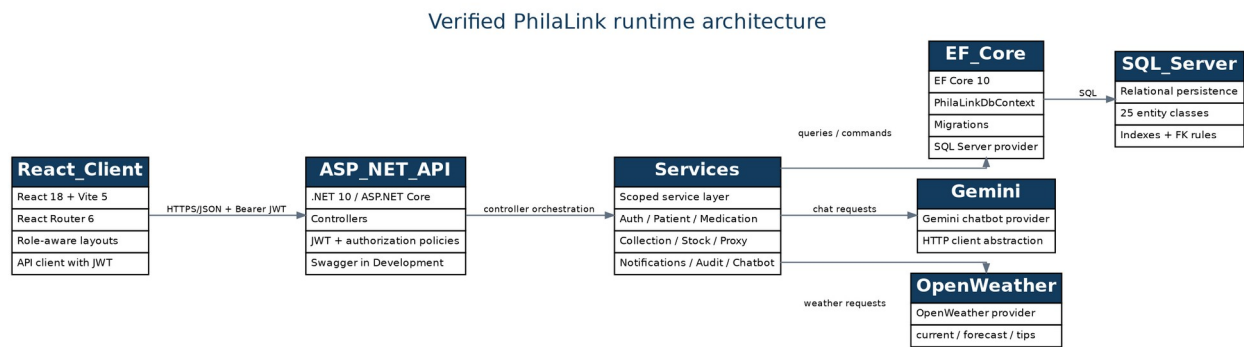


Figure 1. Verified runtime architecture derived from Program.cs, package.json, routing and service structure.

PhilaLink is best described as a role-aware React web client backed by a modular ASP.NET Core API. The backend is a single deployable web application with controllers, scoped services, Entity Framework Core, and external provider abstractions. This is a modular-monolith style architecture rather than a microservice architecture.

Request lifecycle

- A user interacts with a routed React view.
- The frontend API client builds a JSON request and attaches the stored Bearer token for authenticated calls.
- ASP.NET Core authenticates the JWT and evaluates role/policy requirements before controller actions execute.
- Controllers delegate most domain work to scoped services. The medication supply endpoint is a notable controller-level calculation that directly uses PhilaLinkDbContext.
- Services use Entity Framework Core to read or modify SQL Server data.
- Chatbot and weather operations call provider abstractions through configured HttpClient instances.

Technology snapshot

Layer	Verified technology
Frontend	React 18.3.1, Vite 5.4.2, React Router 6.26.2, Tailwind 3.4.10, Lucide React, Leaflet / React-Leaflet, Astra UI packages
Backend	.NET 10, ASP.NET Core Web API, C#, Swagger / OpenAPI
Persistence	Entity Framework Core 10.0.10 with Microsoft.EntityFrameworkCore.SqlServer
Security	JWT Bearer authentication, BCrypt password hashing, role/claim authorization
External integrations	Gemini chatbot provider and OpenWeather HTTP client

3. Frontend architecture

The frontend is a React application built with Vite. It has one browser entry document and uses React Router to render different role-specific views. Calling it a single-page application is technically accurate, but "React web application with client-side routing" is the clearest project description.

Route and layout structure

Role / area	Verified routes and layout
Public	/, /login, /register, /register/verify, /register/success
Password change	/change-password behind PasswordChangeRoute
Patient	/patient with nested routes: medications, appointments, records, clinics, settings; uses PatientAppLayout
Nurse	/nurse, /nurse/patients, /nurse/collections; uses AuthenticatedLayout
Proxy	/proxy, /proxy/patients; uses AuthenticatedLayout
ClinicAdmin / SuperAdmin	/admin, /admin/register-staff, /admin/staff, /admin/audit; SuperAdmin also /admin/clinics and /admin/register-clinic-admin

Frontend security behavior

- ProtectedRoute requires an authenticated session.
- RoleRoute redirects users who do not belong to the allowed role set.
- Accounts with mustChangePassword are forced to /change-password before normal role routes.
- The API client stores the JWT and user payload in localStorage, attaches Authorization: Bearer <token>, and clears the session on HTTP 401.
- API base URL defaults to http://localhost:5174 unless VITE_API_BASE_URL is supplied.

Frontend design note

The browser checks are usability guards, not the security boundary. Backend authorization remains authoritative because client-side routing can be manipulated by a user.

4. Backend architecture

The backend is one ASP.NET Core Web API project. Program.cs composes infrastructure and registers domain services through dependency injection. Controllers expose REST endpoints; services encapsulate business rules; PhilaLinkDbContext models relational persistence.

Registered service domains

Domain service	Responsibility represented in source
AuthService / UserService	registration, authentication, identity-level account operations
PatientService	patient self-service, dashboard, records, preferences and patient-facing aggregation
MedicationService	prescriptions, schedules and adherence logs
MedicationCollectionService	collection workflow and dispensing completion
ClinicStockService / ClinicService	clinic inventory and clinic data
AppointmentService	appointment scheduling workflows
ProxyService	patient-proxy links and proxy-scoped access
NotificationService / AuditLogService	notifications and auditable actions
SymptomAssessmentService	symptom assessment persistence/workflow
ChatbotService + IChatbotProvider	patient chatbot orchestration with provider abstraction
WeatherService	current weather, forecast and health tips
OtpVerificationService	OTP lifecycle and verification

HTTP pipeline facts

- Swagger UI is enabled only when the ASP.NET environment is Development.
- CORS currently uses an AllowAll policy: any origin, method and header. This is permissive and suitable mainly for development unless intentionally retained.
- Authentication executes before authorization.
- The application seeds a SuperAdmin only when no SuperAdmin exists and all required Seed:SuperAdmin* configuration values are present.

5. Authentication, authorization and account state

JWT validation

Control	Current implementation
Issuer	Validated against Jwt:Issuer
Audience	Not validated (ValidateAudience = false)
Signing key	Validated using symmetric key from Jwt:Key
Lifetime	Validated with ClockSkew = 0
Default protected access	Authenticated user with mustChangePassword = false

Authorization policies

Policy	Allowed roles / purpose
SuperAdminOnly	SuperAdmin
AdminOnly	SuperAdmin, ClinicAdmin
ClinicStaff	ClinicAdmin, Nurse
PatientOnly	Patient
ProxyOnly	Proxy
PatientOrProxy	Patient, Proxy
PasswordChangeAllowed	Any authenticated user, including temporary-password state

Account/profile pattern

User is the authentication account. Patient, Nurse, Proxy and Admin are role-specific profile entities connected one-to-one through UserId. This separates credentials and account state from role-specific clinical or administrative attributes.

Security observation

The backend default policy protects normal API usage from temporary-password accounts. Frontend route guards mirror this state, but backend policy enforcement is the decisive control.

6. Functional domains

Domain	Verified capability
Patient self-service	profile/dashboard, medications, records, appointments, collections, notifications, preferences, clinics, assigned worker context
Appointments	create/book, reschedule and cancel patient appointments with status values persisted in Appointment
Medication adherence	medication records, daily schedules, taken/skipped logs
Medication supply	patient supply endpoint estimates dispensed quantity remaining and days remaining when structured dosing data exists
Collections and stock	scheduled medication collections, collection items tied to prescription and exact clinic stock, stock decrement on completion
Proxy care	ProxyLink connects proxy and patient with assignment metadata and active/ended lifecycle
Clinic administration	clinic records, nurses/admins, stock, audit scope
Notifications	user notifications with read state
Health data	allergies, conditions, metrics, records, symptom assessments, preferences
Chatbot	persisted conversations/messages plus Gemini provider abstraction
Weather	OpenWeather-backed current/forecast/tips routes

Architectural boundary note

Weather readings are not persisted as a dedicated entity in the current model. Chatbot conversations are persisted. Medication supply is returned through MedicationSupplyDto and is derived from persisted medication, schedule and collection data rather than stored as its own table.

7. Medication collection and supply architecture

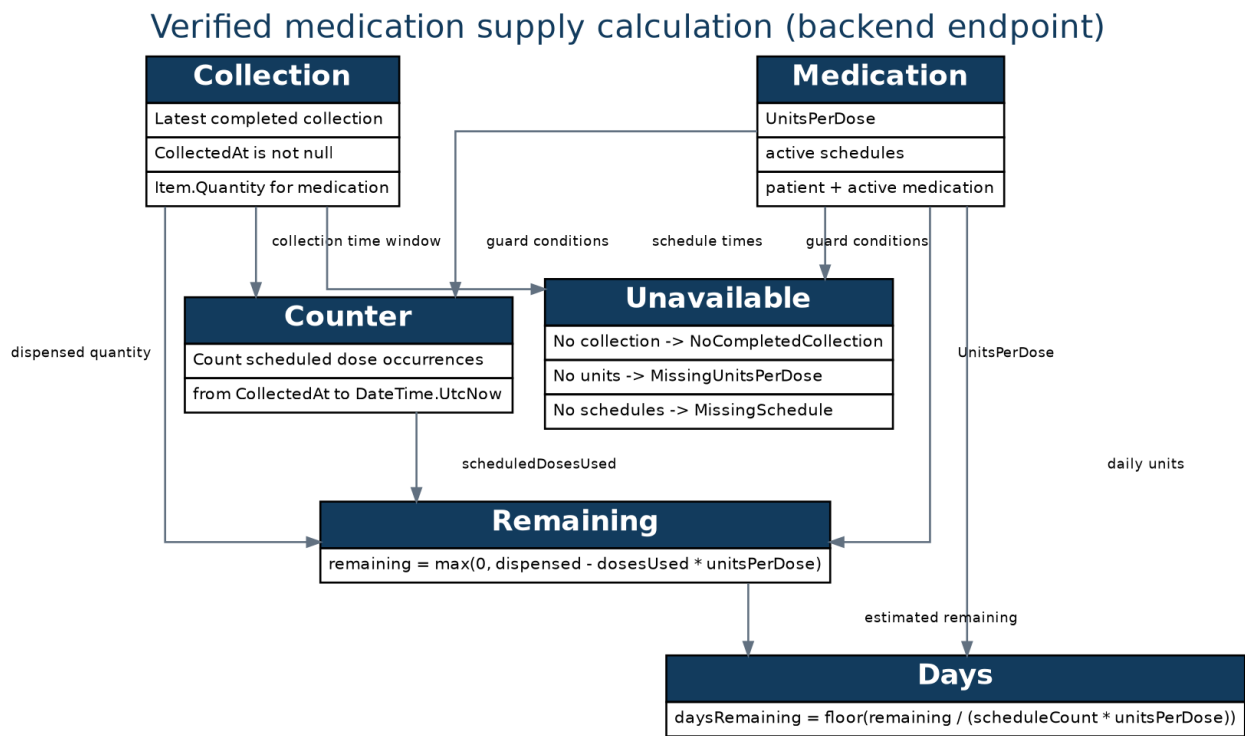


Figure 2. Supply calculation implemented by MedicationSupplyController.

The supply calculation is a derived estimate, not a stock balance column on Medication. For each active patient medication, the controller selects the latest completed collection that contains an item for that medication. It sums the item quantity in that collection, counts scheduled dose occurrences from CollectedAt through the current UTC time, and multiplies them by UnitsPerDose.

Formula and states

Output	Rule
EstimatedRemainingQuantity	$\max(0, \text{dispensedQuantity} - \text{scheduledDosesUsed} \times \text{UnitsPerDose})$
DaysRemaining	$\text{floor}(\text{EstimatedRemainingQuantity} / (\text{activeScheduleCount} \times \text{UnitsPerDose}))$
NoCompletedCollection	No completed collection exists for that medication
MissingUnitsPerDose	Collection exists but structured UnitsPerDose is absent or non-positive
MissingSchedule	Collection and UnitsPerDose exist but there is no active schedule
Available	All inputs are available and the estimate is calculated

Precision

Medication.UnitsPerDose is explicitly configured as decimal(18,4), so fractional dosing such as 0.5 units can be represented without relying on free-text dosage parsing.

8. Entity relationship model

The data model is presented as four domain ERDs instead of one global graph. This is deliberate: a single 25-entity drawing creates a web of crossing connectors that is difficult to audit. Each diagram keeps attributes inside its entities, while the relationship catalogue later in this document records exact cardinalities and optionality.

Diagram conventions

- PK marks the primary key.
- FK marks a persisted foreign key.
- A trailing ? means the field is nullable/optional.
- Connectors show persisted relationships only. Derived DTOs and computed supply values are not database entities.
- Large address, emergency-contact and audit groups are compressed in the ERD for legibility; the entity dictionary lists the complete scalar fields.

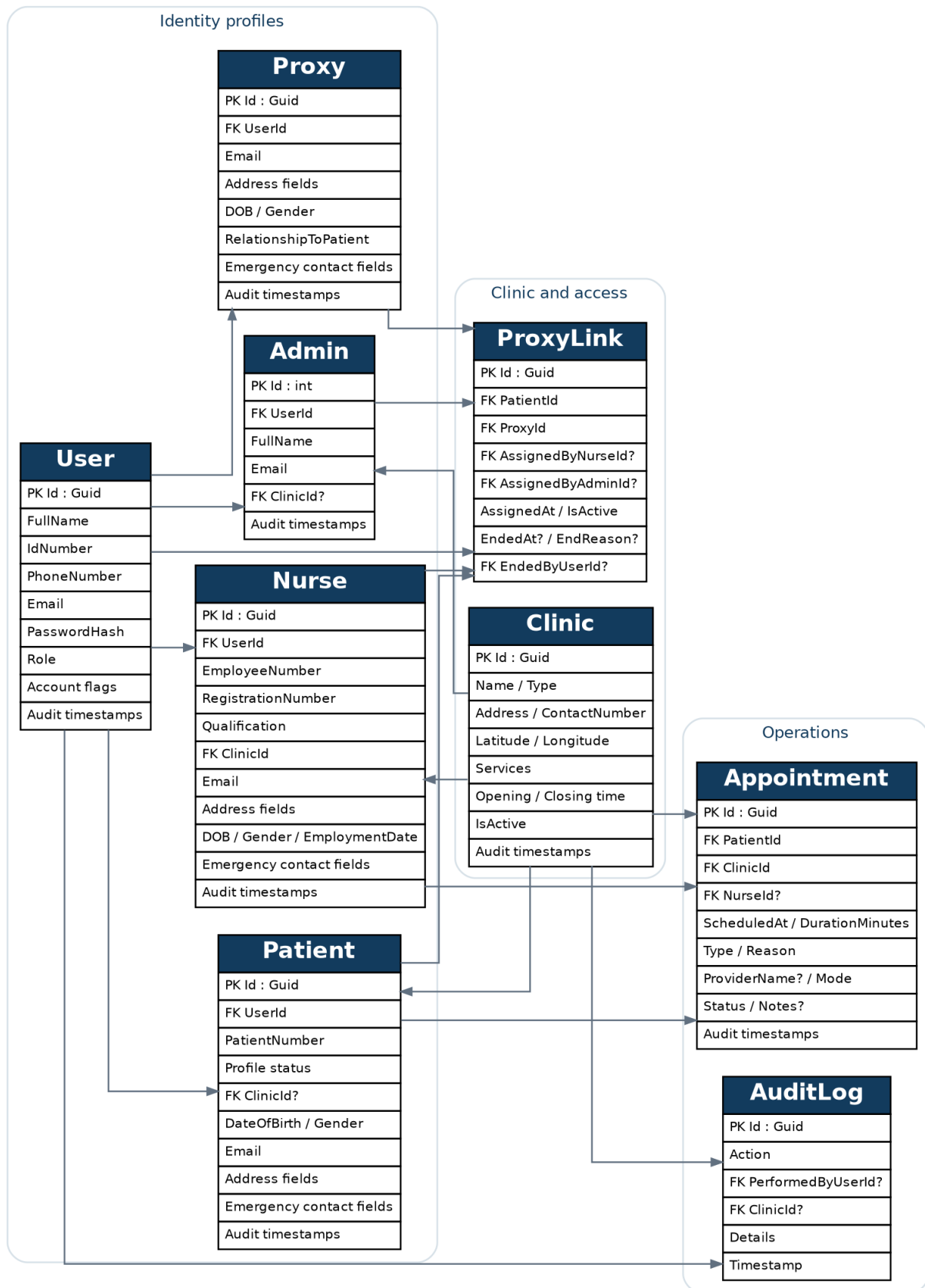
ERD pages

ERD	Coverage
ERD 1	Identity, role profiles, clinic scope, proxy links, appointments and audit
ERD 2	Patient clinical data, preferences and chatbot history
ERD 3	Medication, schedules, adherence, collection workflow and clinic stock
ERD 4	Notifications, OTP verification, audit context and conversation persistence

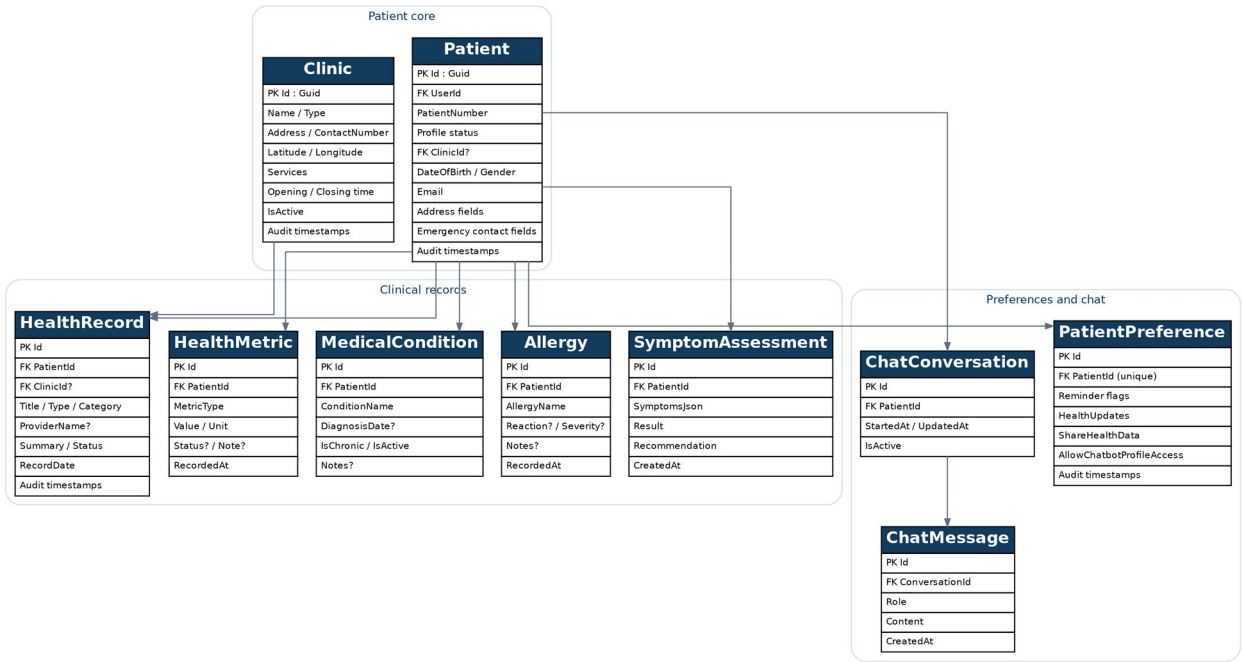
Why the connectors are simpler in this revision

Relationship labels were removed from the connector lines and moved into the relationship catalogue. This prevents text-on-line collisions and makes each path easier to trace in portrait mode.

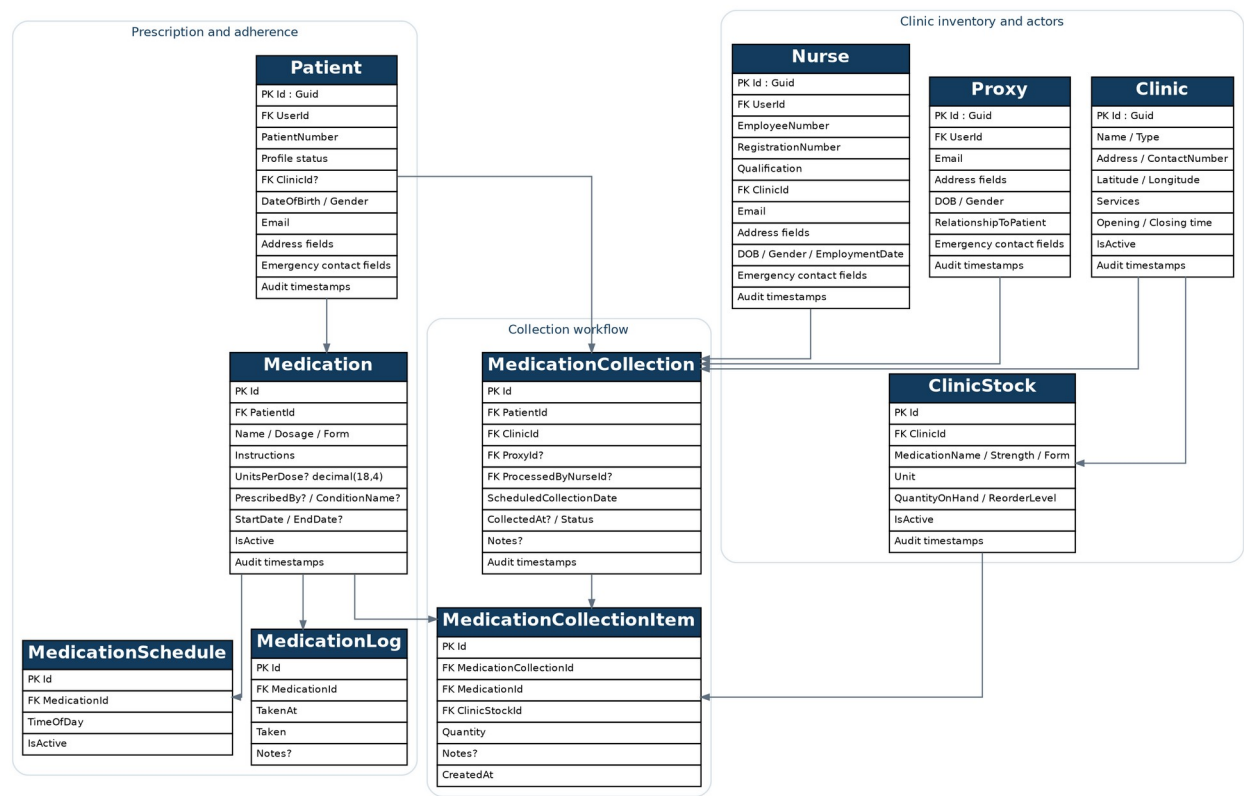
ERD 1 - Identity, clinic scope, proxy links and appointments



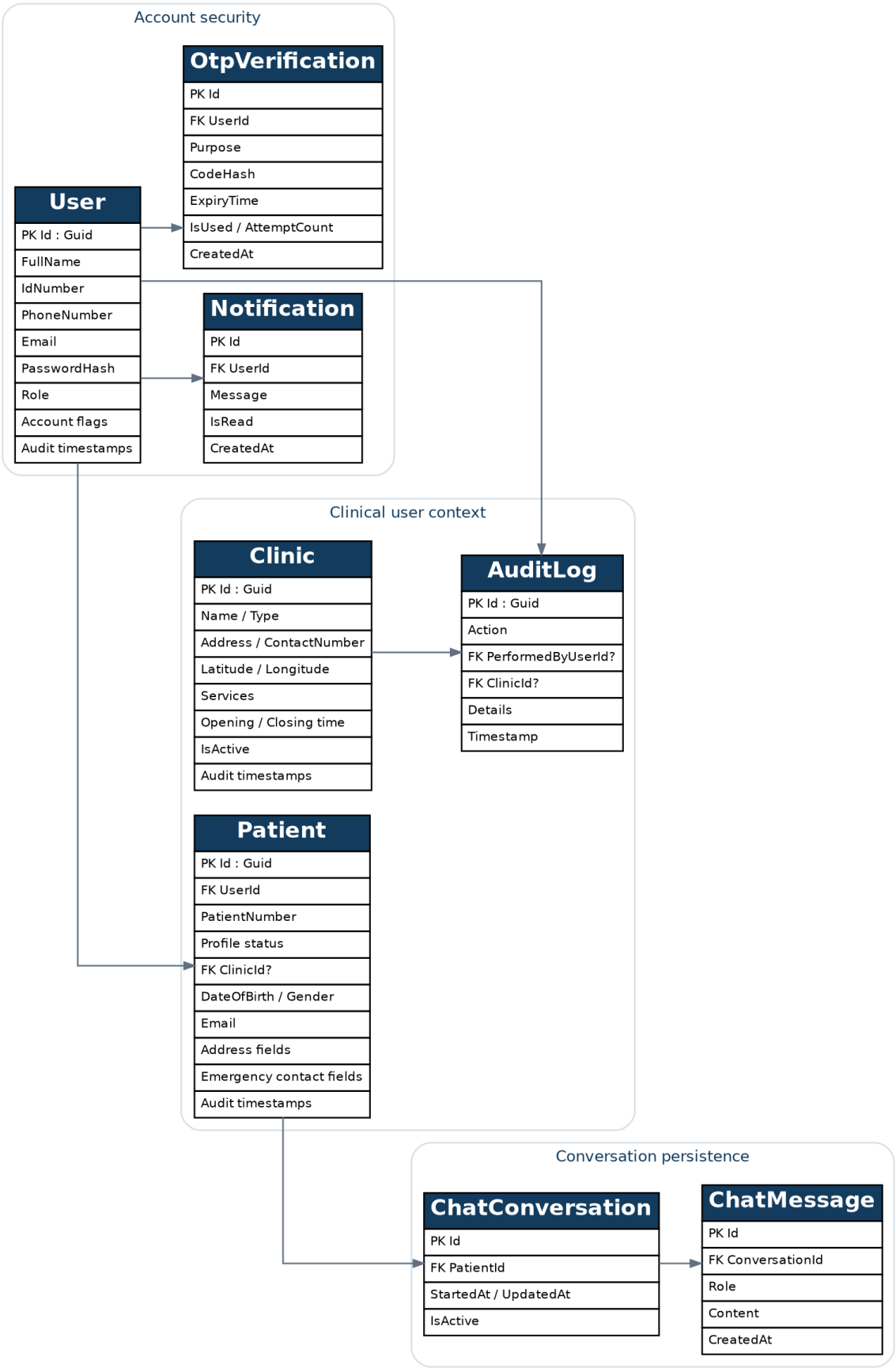
ERD 2 - Patient clinical data, preferences and chatbot history



ERD 3 - Medication, dispensing, collection and stock



ERD 4 - Notifications, OTP, audit and conversation persistence



9. Entity dictionary - identity and clinic scope

Entity	Persisted scalar attributes
User	PK Id : Guid; FullName; IdNumber; PhoneNumber; Email; PasswordHash; Role; IsActive; IsVerified; VerifiedAt?; MustChangePassword; CreatedAt; UpdatedAt?
Patient	PK Id : Guid; FK UserId; PatientNumber; IsProfileComplete; ProfileCompletedAt?; FK ClinicId?; DateOfBirth; Gender; Email; AddressLine1/2; Suburb / City / Province / PostalCode; EmergencyContactName; EmergencyContactPhone; EmergencyContactRelationship; CreatedAt; UpdatedAt?
Nurse	PK Id : Guid; FK UserId; EmployeeNumber; RegistrationNumber; Qualification; FK ClinicId; Email; AddressLine1/2; Suburb / City / Province / PostalCode; DateOfBirth; Gender; EmploymentDate; Emergency contact fields; CreatedAt; UpdatedAt?
Proxy	PK Id : Guid; FK UserId; Email; AddressLine1/2; Suburb / City / Province / PostalCode; DateOfBirth; Gender; RelationshipToPatient; Emergency contact fields; CreatedAt; UpdatedAt?
Admin	PK Id : int; FK UserId; FullName; Email; FK ClinicId?; CreatedAt; UpdatedAt?
Clinic	PK Id : Guid; Name; Type; Address; ContactNumber; Latitude; Longitude; Services; OpeningTime?; ClosingTime?; IsActive; CreatedAt; UpdatedAt?
ProxyLink	PK Id : Guid; FK PatientId; FK ProxyId; FK AssignedByNurselId?; FK AssignedByAdminId?; AssignedAt; IsActive; EndedAt?; FK EndedByUserId?; EndReason?
Appointment	PK Id : Guid; FK PatientId; FK ClinicId; FK NurseId?; ScheduledAt; DurationMinutes; Type; Reason; ProviderName?; Mode; Status; Notes?; CreatedAt; UpdatedAt?

10. Entity dictionary - clinical data

Entity	Persisted scalar attributes
AuditLog	PK Id : Guid; Action; FK PerformedByUserId?; FK ClinicId?; Details; Timestamp
Allergy	PK Id : Guid; FK PatientId; AllergyName; Reaction?; Severity?; Notes?; RecordedAt
MedicalCondition	PK Id : Guid; FK PatientId; ConditionName; DiagnosisDate?; IsChronic; Notes?; IsActive
HealthMetric	PK Id : Guid; FK PatientId; MetricType; Value; Unit; Status?; Note?; RecordedAt
HealthRecord	PK Id : Guid; FK PatientId; FK ClinicId?; Title; Type; Category; ProviderName?; Summary; Status; RecordDate; CreatedAt; UpdatedAt?
PatientPreference	PK Id : Guid; FK PatientId (unique); MedicationReminders; AppointmentReminders; ClinicNotifications; HealthUpdates; ShareHealthData; AllowChatbotProfileAccess; CreatedAt; UpdatedAt?
SymptomAssessment	PK Id : Guid; FK PatientId; SymptomsJson; Result; Recommendation; CreatedAt

11. Entity dictionary - medication, communication and security

Entity	Persisted scalar attributes
Medication	PK Id : Guid; FK PatientId; Name; Dosage; Form; Instructions; UnitsPerDose? decimal(18,4); PrescribedBy?; ConditionName?; StartDate; EndDate?; IsActive; CreatedAt; UpdatedAt?
MedicationSchedule	PK Id : Guid; FK MedicationId; TimeOfDay; IsActive
MedicationLog	PK Id : Guid; FK MedicationId; TakenAt; Taken; Notes?
MedicationCollection	PK Id : Guid; FK PatientId; FK ClinicId; FK ProxyId?; FK ProcessedByNurseId?; ScheduledCollectionDate; CollectedAt?; Status; Notes?; CreatedAt; UpdatedAt?
MedicationCollectionItem	PK Id : Guid; FK MedicationCollectionId; FK MedicationId; FK ClinicStockId; Quantity; Notes?; CreatedAt
ClinicStock	PK Id : Guid; FK ClinicId; MedicationName; Strength; Form; Unit; QuantityOnHand; ReorderLevel; IsActive; CreatedAt; UpdatedAt?
Notification	PK Id : Guid; FK UserId; Message; IsRead; CreatedAt
OtpVerification	PK Id : Guid; FK UserId; Purpose; CodeHash; ExpiryTime; IsUsed; AttemptCount; CreatedAt
ChatConversation	PK Id : Guid; FK PatientId; StartedAt; UpdatedAt; IsActive
ChatMessage	PK Id : Guid; FK ConversationId; Role; Content; CreatedAt

Not entities

DTOs such as MedicationSupplyDto, PatientDashboardDto, response DTOs and weather DTOs are API contracts or derived projections. They are intentionally excluded from the ERD because they do not represent independent persisted tables.

12. Relationship catalogue

Principal(s)	Dependent	Cardinality	Meaning
User	Patient / Nurse / Proxy / Admin	1 to 0..1 each	Role-specific profile via unique UserId
Clinic	Patient / Nurse / Admin	1 to many	Patient/Admin clinic FK optional; Nurse clinic required
Patient	Allergy, Condition, Metric, Record, SymptomAssessment	1 to many	Clinical history / measurements
Patient	PatientPreference	1 to 0..1	Unique dependent PatientId
Patient	Medication	1 to many	Patient prescriptions
Medication	Schedule / Log	1 to many	Daily dosing schedule and adherence entries
Patient + Clinic	MedicationCollection	1 to many	Collection belongs to patient and clinic
Proxy / Nurse	MedicationCollection	0..1 per collection	Optional proxy and processing nurse
MedicationCollection	MedicationCollectionItem	1 to many	Dispensed line items
Medication + ClinicStock	MedicationCollectionItem	1 to many	Item references prescription and exact stock row
Patient + Proxy	ProxyLink	many-to-many over time	Link entity stores assignment lifecycle
Patient + Clinic + Nurse	Appointment	Patient/Clinic required; Nurse optional	Appointment scheduling
User	Notification / OtpVerification	1 to many	User communications and verification attempts
Patient	ChatConversation	1 to many	Persisted chatbot sessions
ChatConversation	ChatMessage	1 to many	Conversation messages
User / Clinic	AuditLog	optional many	Audit context can be system-wide, user-linked and/or clinic-linked

Delete behavior and integrity

DbContext explicitly configures restrictive deletes for many cross-domain links such as User profiles, clinic assignments, appointments, collections and stock references, while selected child data uses cascade semantics, for example medication schedules/logs and patient preferences. The model also defines uniqueness/index rules, including unique user identity/phone fields, unique role profile UserId mappings, a unique PatientPreference.PatientId, and a unique collection-item pair per collection and medication.

13. Repository truth, limitations and current development status

Verified repository mismatches

Area	What source code says	What documentation says / implication
Database provider	UseSqlServer + EF Core SQL Server package	README says PostgreSQL/Npgsql/Supabase target. Treat README section as outdated.
Frontend application type	React app with client-side routing and one index.html	"SPA" is a valid technical label, but can be misread as a one-screen product. This report uses more explicit wording.
Medication supply frontend	Backend endpoint and migrations are checked in	Project chat shows frontend dashboard wiring is still the next task; do not mark complete until committed and tested.

Current development checkpoint from project chat

- Completed: structured UnitsPerDose and decimal precision migration.
- Completed: medication supply backend endpoint and runtime verification.
- In progress: surface accurate medication supply on the patient dashboard/front end.
- Next planned: notification refresh/state handling, loading/error/empty-state audit, responsive/orientation audit and final regression build.

Source files used for this verification

- Backend: Program.cs; PersonalProject.csproj; Data/PhilaLinkDbContext.cs; Models/Entities/*.cs; Controllers/MedicationSupplyController.cs; Services/Interfaces/*; Services/Implementations/*; repository tree.
- Frontend: package.json; src/App.jsx; src/routes/ProtectedRoute.jsx; src/services/api/client.js; repository tree.
- Project chat: current TODO and runtime/build confirmations only where not contradicted by GitHub.

How to use this document

Treat this PDF as a source-aligned architecture snapshot, not a substitute for code. When a future migration, entity or route changes, regenerate the ERD and the verification table from the updated default branches.